Parallel and Distributed Systems

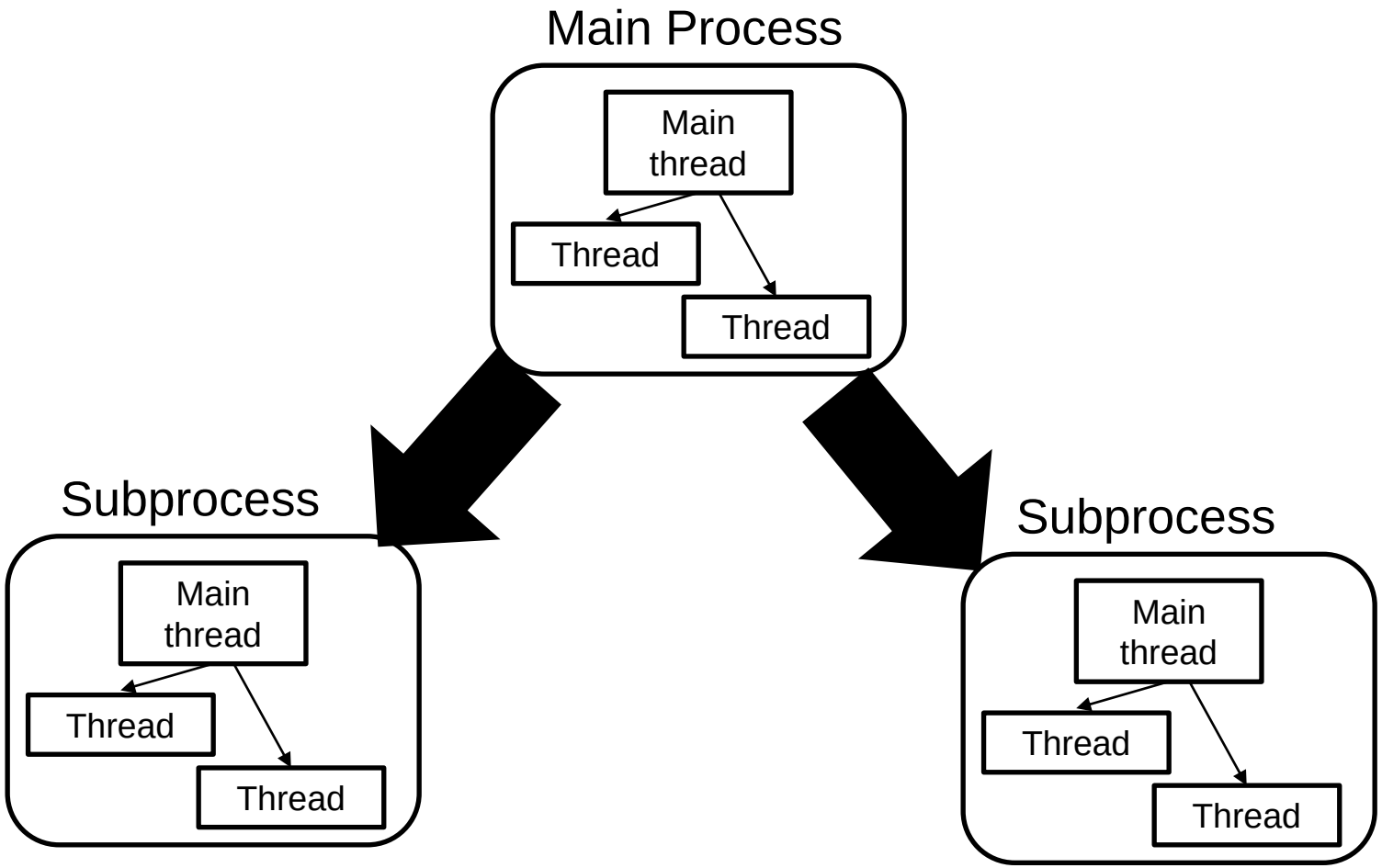
Chapter 4: Inter-Process Communication

Prof. Dr. Martin Sträßer

Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap: Structure of Concurrent Programs



Recap: Processes

- Entity that encapsulates an executing application
- Spawned and controlled by the operating system
- **Two (unrelated) processes are fully isolated from each other**
  - Need for dedicated communication mechanisms between them

- Brainstorming Question
  - How would you realize inter-process communication?
  - Possible ideas: Mailbox, shared whiteboard, conveyor belt

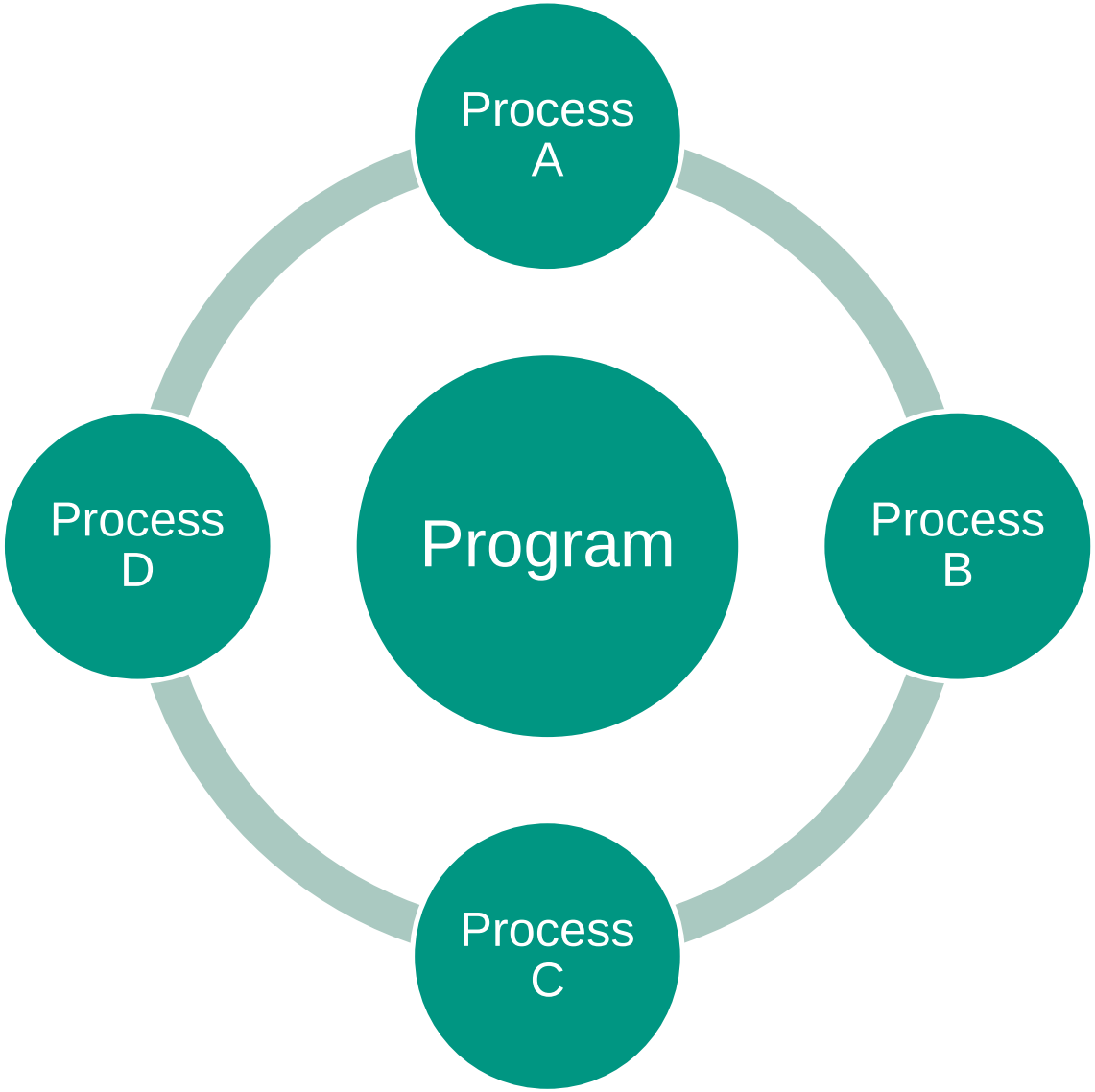
Overview

- IPC Terminology
- Shared Memory IPC
- Message-Passing IPC
- IPC Design
- IPC Use Cases
  - Parallel Computation
  - Producer-Consumer Pipelines
  - Microservices

IPC Terminology

Inter-Process Communication

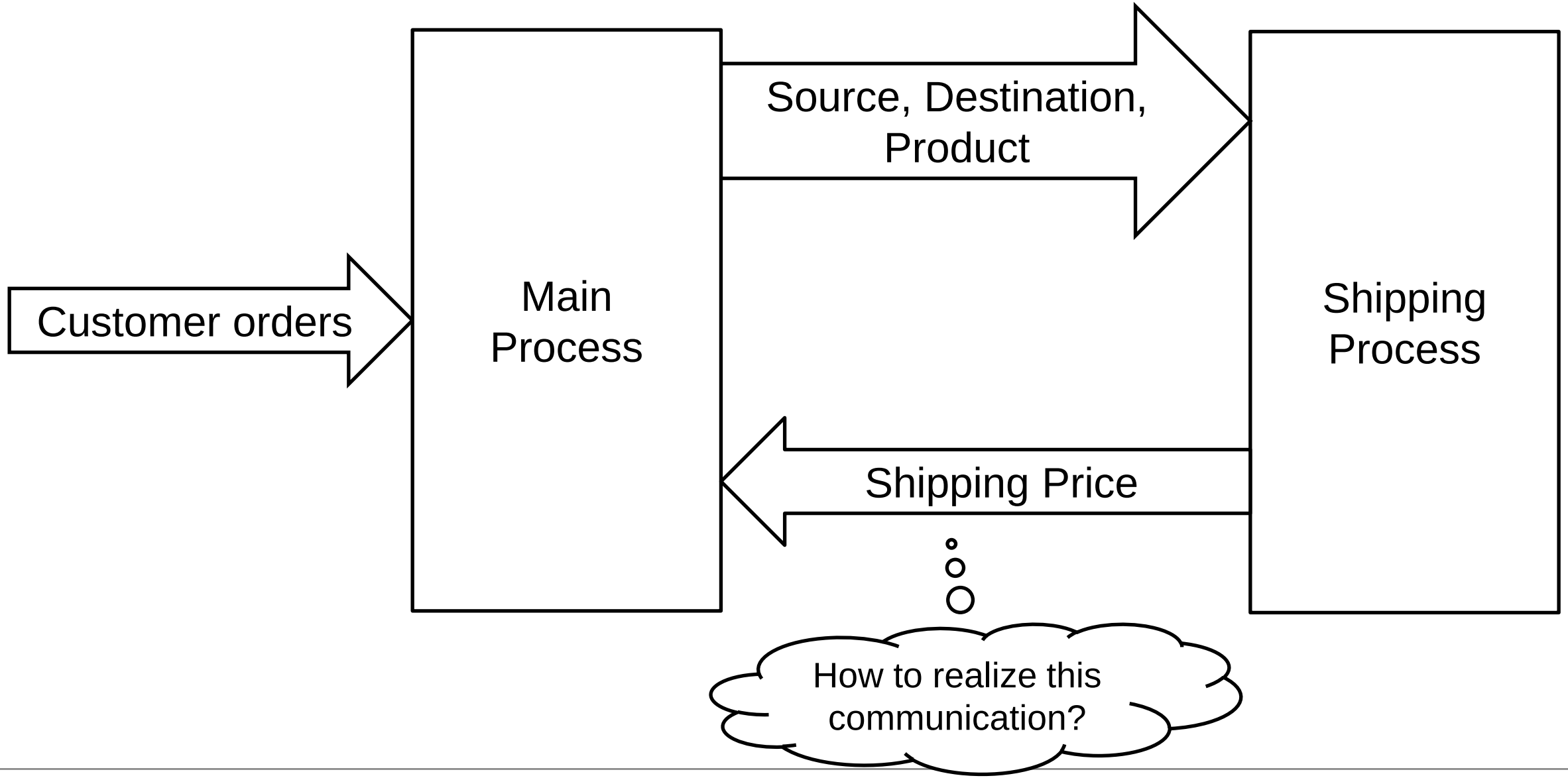
- When we develop a concurrent program with multiple processes, these processes do not share variables/memory by default
- However, these processes may still fulfill a common goal
- In practice, processes usually need to **exchange data** or **synchronize their actions**
- Focus of this chapter: Data exchange of processes running on the same machine



Example Application

- We develop an international online store, where customers can order products autonomously
- When customers submit their orders, we need to:
  - Check whether we have the ordered products in stock
  - Read and process customer credentials
  - Process payments
  - Calculate shipping prices
- To make the order processing faster, we decide to outsource the shipping price calculation to a new process
- As we receive many orders, we do not want to start a new process for every order instead we have a continuously running process (= service)

Example Application

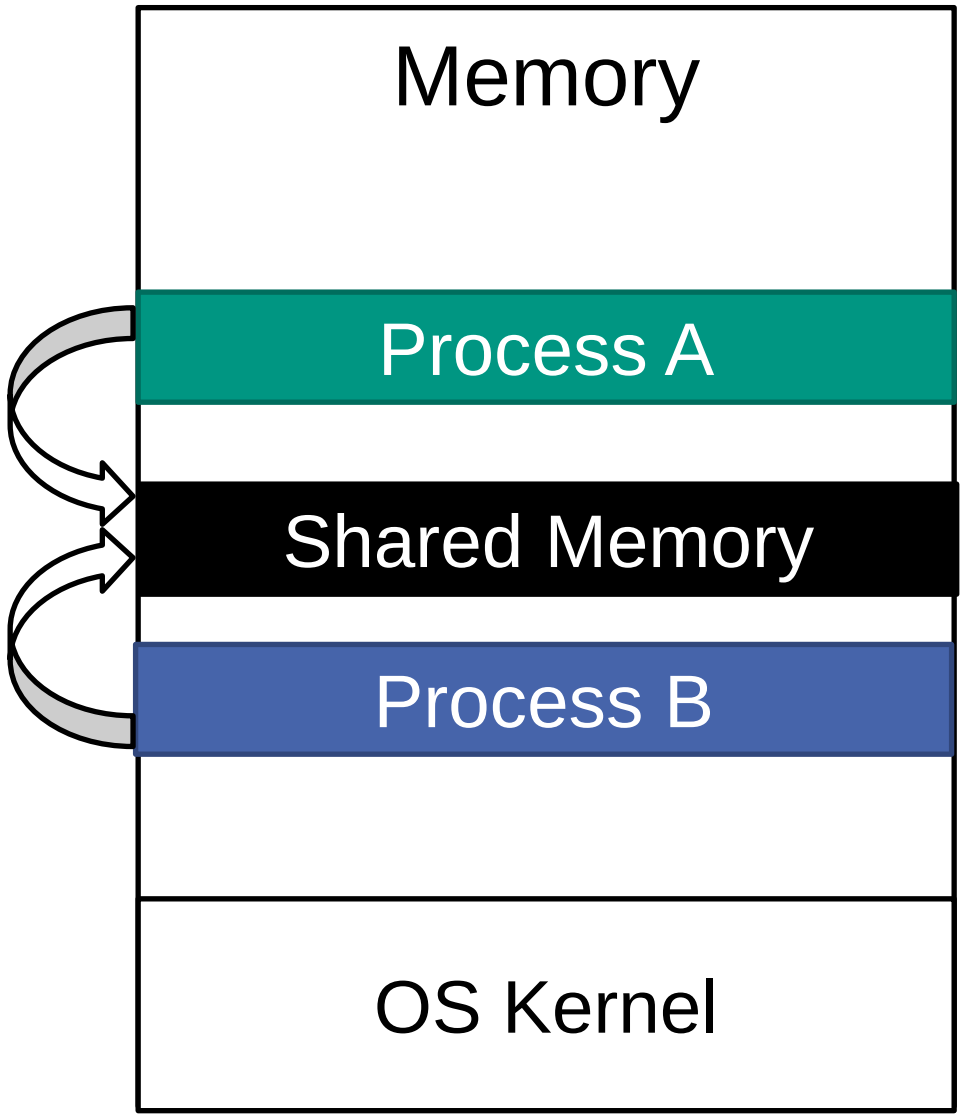


IPC Models

- There are two fundamental models for IPC
  - **Shared Memory**
  - **Message Passing**

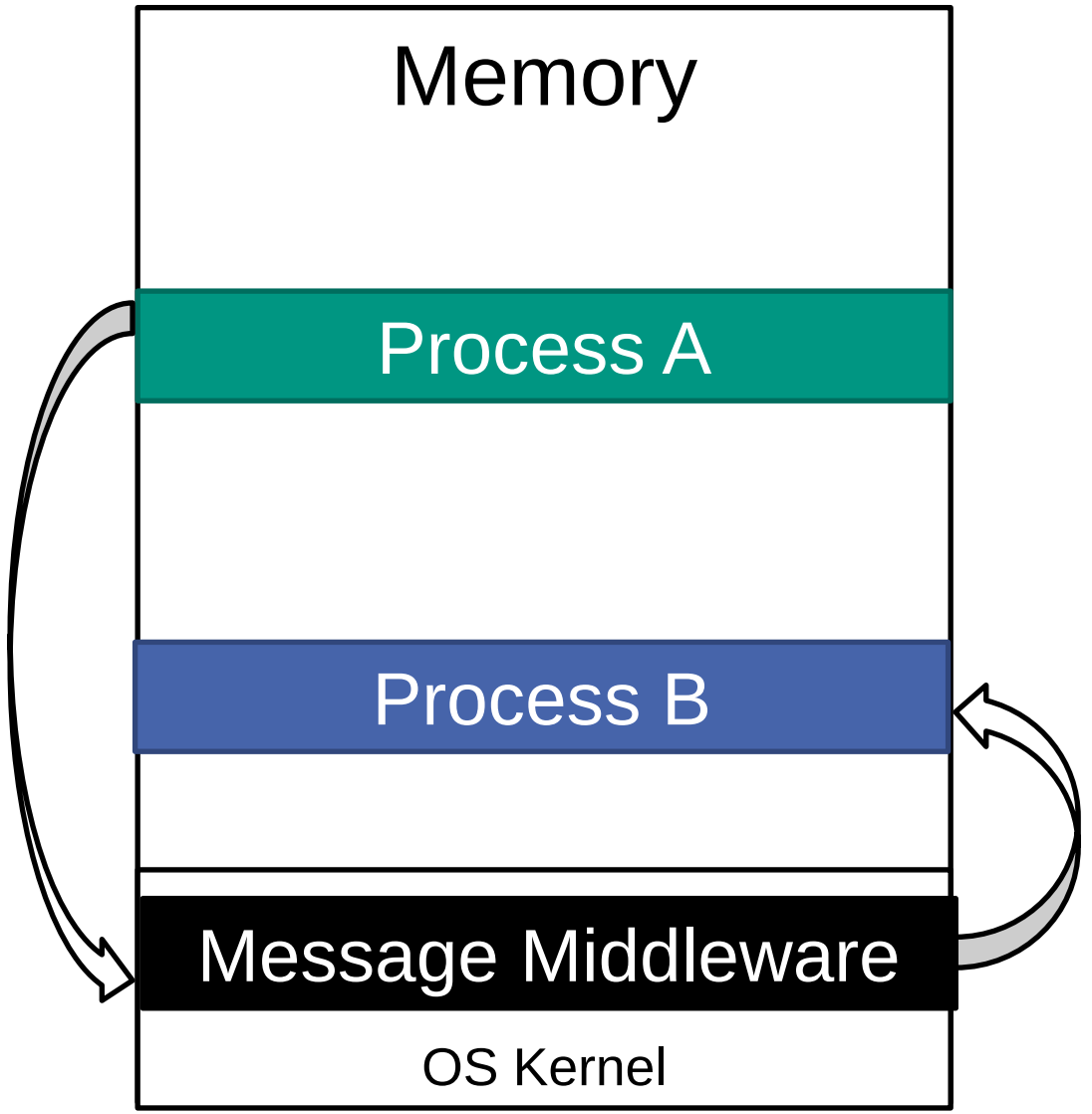
Shared Memory

- Multiple processes are given access to the same region of memory
- Communication via direct reads and writes to the memory region
- OS kernel typically not involved in communication
- Implementation in high-level languages via global variables or OS libraries



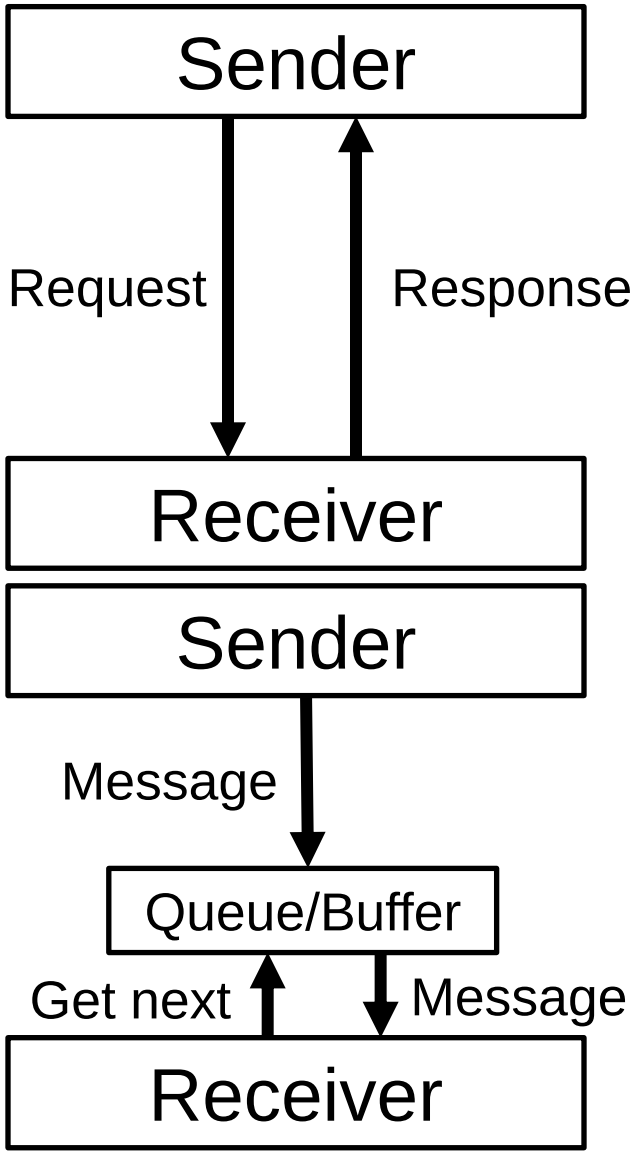
Message Passing

- Communication via sending and receiving of messages (fixed or variable size)
- Needs some „construct“ in the middle that transports messages (*middleware*)
- Message-passing IPC on one machine usually requires kernel for communication
- Implementation in high-level languages often via pipes or message queues



Synchronous vs. Asynchronous Communication

- Synchronous Communication
  - Sender and receiver perform coordination in time
  - A send is completed only when the receiver is ready
  - From the view of the sender: Guaranteed that the message has been received (and processed)
- Asynchronous Communication
  - Loose coupling in time
  - The sender does not wait for the receiver to be ready
  - Messages may be queued, buffered, and handled later



Blocking vs. Non-Blocking IPC

- Describes the behavior of a process during communication
- Blocking IPC
  - Process is suspended until operation completes
  - No useful operation during the wait
- Non-Blocking IPC
  - IPC operation returns immediately
  - Completion is checked later (polling, callback, future)

Non-Blocking IPC

- Scenario: We send out a message and until we receive a response, we perform some other work. How can the response handling be realized?
- Analogy: We order something in a restaurant. In the meantime, we talk the our mates. How to determine whether our food is ready?

Polling

- Option 1: Polling
  - We ask in regular intervals whether the response has been completed
  - No automatic notification
- Analogy: We request the waiter every minute and ask whether our food is ready. The waiter brings the food only on request.
- Pros
  - Very simple
  - Easy timing control
- Cons
  - CPU wastage
  - Increased latency (depends on polling interval)

Callback Functions

- Option 2: Callback function
  - When executing the IPC call, we register a callback function that is executed when the response arrives
  - See also: Reactive programming
- Analogy: Waiter calls our name when our food is ready
- Pros
  - No blocking
  - No polling
- Cons
  - No defined execution order
  - Hard error handling
  - Often assumes implicitly that response arrives „soon“

Futures/Promises

- Option 3: Futures/Promises
  - The return value of the non-blocking IPC operation is a future, a specific data type
  - A future is like a box that we can treat as a normal variable
  - Once we need the response, we „open the box“
  - If the response is there, we extract it and continue, if not, we wait until the response is there
- Analogy: We get an order number from the waiter. When we ended our conversation with our friends, we go to the pick-up station. We may wait there, if our food is not ready.

Futures/Promises

- Pros
  - Structured, readable code
  - Clean error handling
  - Used in many modern programming languages
- Cons
  - Can lead to hidden blocking behavior, if we open our boxes „too early“

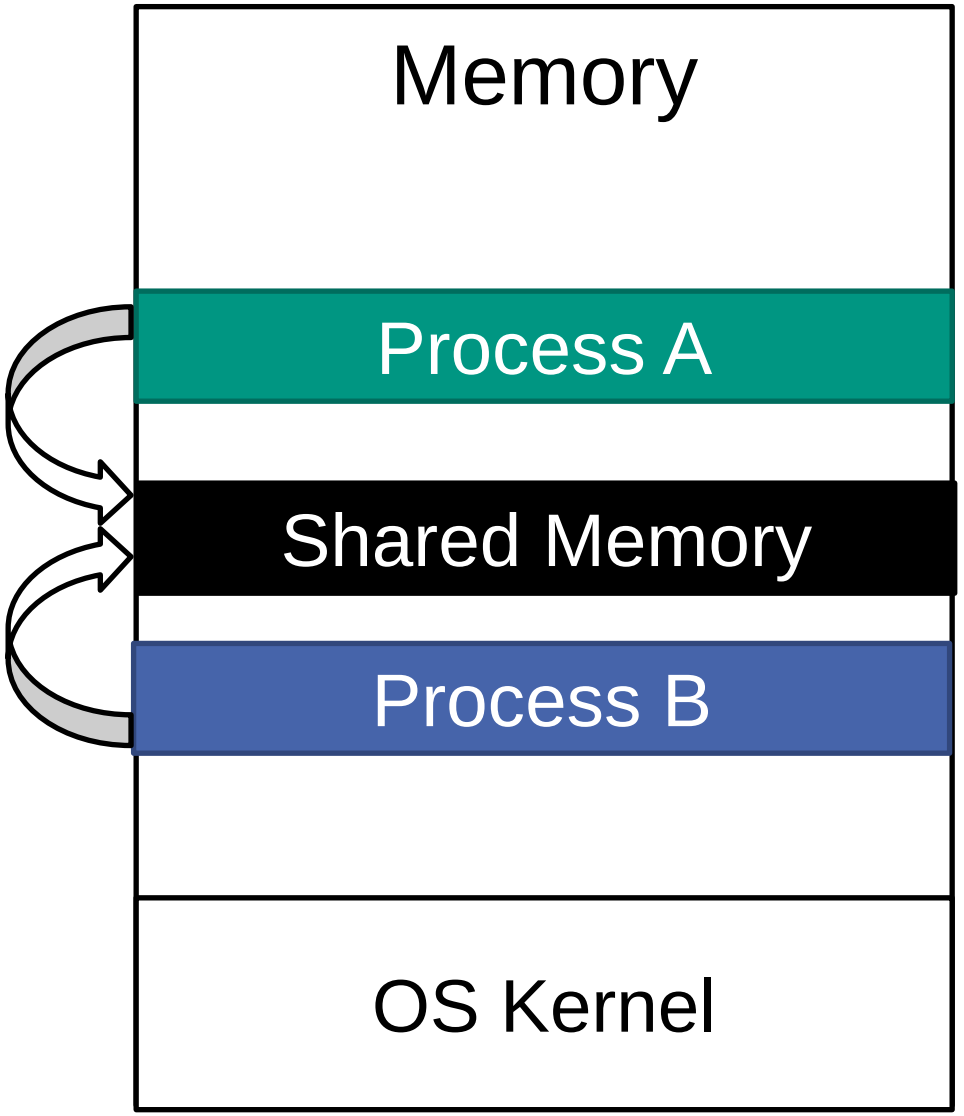
Sync, Async, Blocking, Non-Blocking

- Synchronous and asynchronous communication describe a communication style
- Blocking and non-blocking describe the control flow of a process during an IPC operation
- They can be combined
  - Sync + blocking: Classic request-response scheme
  - Sync + non-blocking: Futures
  - Async + blocking: Receiver waiting for message
  - Async + non-blocking: Receiver uses callback function to receive messages

Shared Memory IPC

Shared Memory

- Multiple processes are given access to the same region of memory
- Communication via direct reads and writes to the memory region
- OS kernel typically not involved in communication
- Implementation in high-level languages via global variables or OS libraries



Shared Memory

- Steps that are performed (in the background):
  - Request for a shared memory segment by the main/parent process
  - OS kernel **creates** segment and assigns unique identifier
  - Parent process passes identifier to child processes
  - Processes **attach** to shared memory
  - **Synchronized access** to shared memory by the processes
  - Processes **detach** from shared memory
  - Memory segment is **deleted** if no process attached anymore
- In low-level languages like C, those steps can be realized with system calls
- In high-level languages, libraries perform these tasks in the background (e.g., Python multiprocessing)

Shared Memory in Python

- Question: What will be the output of the program?

```python
from multiprocessing import Process

x = 0

def task():  1 usage
    global x
    x = x + 1

if __name__ == "__main__":
    process = Process(target=task)
    process.start()
    process.join()
    print(x)
```

See code no. 1

Multiprocessing Value and Array

- Reminder: Python Processes do not share memory by default, every process has its own address space and interpreter
- Easy way: Value and Array classes from the multiprocessing library
- They live on the boundary of Python classes and C variables (they are wrappers for C shared memory variables)

```python
from multiprocessing import Process, Value, Array

val = Value( typecode_or_type: "i",  *args: 0)
arr = Array( typecode_or_type: "i",  size_or_initializer: 4)

def task(val, arr):  1 usage
    val.value = val.value + 1
    for i in range(len(arr)):
        arr[i] = arr[i] + 1


if __name__ == "__main__":
    process = Process(target=task, args=(val, arr))
    process.start()
    process.join()
    print(val.value)
    for i in range(len(arr)):
        print(arr[i])
```

See code no. 2

Multiprocessing Value and Array

```python
from multiprocessing import Process, Value, Array

val = Value( typecode_or_type: "i",  *args: 0)
arr = Array( typecode_or_type: "i",  size_or_initializer: 4)

def task(val, arr):  1 usage
    val.value = val.value + 1
    for i in range(len(arr)):
        arr[i] = arr[i] + 1


if __name__ == "__main__":
    process = Process(target=task, args=(val, arr))
    process.start()
    process.join()
    print(val.value)
    for i in range(len(arr)):
        print(arr[i])
```

Main process creates shared memory segment

Main process passes identifier/access info to child process

Synchronized access to shared memory (process-safe mutex activated by default)

See code no. 2

Multiprocessing Value and Array

Type identifier
(C data type):
i – int
d – double
f – float
b – raw bytes
(e.g., for strings)
…

```python
from multiprocessing import Process, Value, Array

val = Value( typecode_or_type: "i",  *args: 0)
arr = Array( typecode_or_type: "i",  size_or_initializer: 4)

def task(val, arr):  1 usage
    val.value = val.value + 1
    for i in range(len(arr)):
        arr[i] = arr[i] + 1


if __name__ == "__main__":
    process = Process(target=task, args=(val, arr))
    process.start()
    process.join()
    print(val.value)
    for i in range(len(arr)):
        print(arr[i])
```

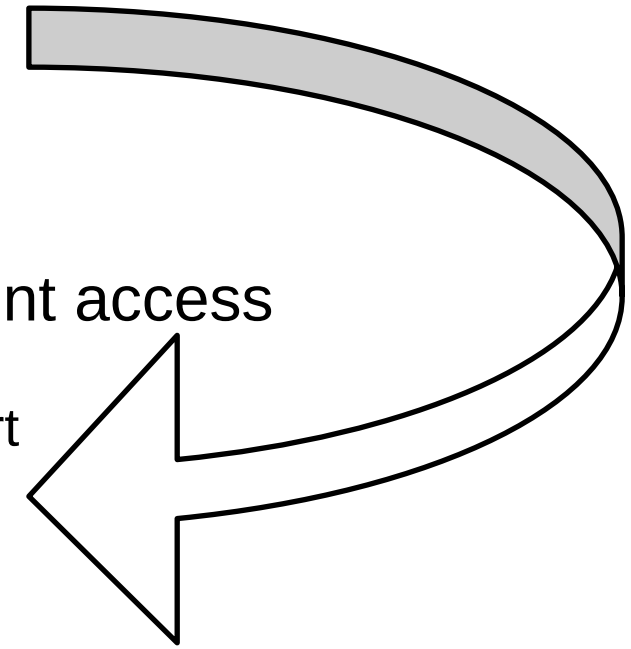
Initial value

Size of array

See code no. 2

Multiprocessing Value and Array

- Advantages:
  - Minimal overhead
  - Good for large shared numeric data
  - Good for high-frequency updates
- Disadvantages:
  - Fixed size
  - Not resizeable
  - Bad for objects
  - Bad for infrequent access



Can we still support strings or complex objects somehow?

General-Purpose Shared Memory

- Python also supports C-like low-level access to shared memory segments with multiprocessing's shared_memory
- Caution, by default there is:
  - No defined data type / memory structure
  - No access synchronization
- We interpret memory content however we like

```python
from multiprocessing import Process, shared_memory

def task(shm_name: str):  1 usage
    shm = shared_memory.SharedMemory(name=shm_name)
    shm.buf[0:5] = b'hello'
    shm.close()

if __name__ == "__main__":
    shm = shared_memory.SharedMemory(create=True, size=10)

    p = Process(target=task, args=(shm.name,))
    p.start()
    p.join()

    print(bytes(shm.buf[:5]))

    shm.close()
    shm.unlink()
```

See code no. 3

General-Purpose Shared Memory

```python
from multiprocessing import Process, shared_memory

def task(shm_name: str):  1 usage
    shm = shared_memory.SharedMemory(name=shm_name)
    shm.buf[0:5] = b'hello'
    shm.close()

if __name__ == "__main__":
    shm = shared_memory.SharedMemory(create=True, size=10)

    p = Process(target=task, args=(shm.name,))
    p.start()
    p.join()

    print(bytes(shm.buf[:5]))

    shm.close()
    shm.unlink()
```

Main process creates shared memory segment

Main process passes identifier/access info to child process

Un-synchronized access to shared memory (not process-safe)

See code no. 3

General-Purpose Shared Memory

```python
from multiprocessing import Process, shared_memory

def task(shm_name: str):  1 usage
    shm = shared_memory.SharedMemory(name=shm_name)
    shm.buf[0:5] = b'hello'
    shm.close()

if __name__ == "__main__":
    shm = shared_memory.SharedMemory(create=True, size=10)

    p = Process(target=task, args=(shm.name,))
    p.start()
    p.join()

    print(bytes(shm.buf[:5]))

    shm.close()
    shm.unlink()
```

Child process detaches from shared memory

Main process detaches and deletes shared memory (if we forget this: memory leak)

Raw bytes are written/read (string objects would need encoding)

See code no. 3

Comparison

Feature	Multiprocessing Value/Array	Multiprocessing shared_memory
Abstraction level	High	Low
Data types	Can be specified	Raw bytes only
Synchronized access	Yes by default (can be deactivated)	No
Variable size	No	No
Performance	Fast	Fastest
Usage with other performant libraries like NumPy	Not recommended	Supported

(File-Based) Memory Mapping

- The previous examples of shared memory used RAM storage only
- mmap is an established way of involving disk storage
- Similarities to shared memory
  - No access synchronization, data types, or variable size
- Differences to shared memory
  - Data is persisted on disk, good if RAM storage is rare or shared data is huge
  - Still very fast especially for reads (good caching), but writes may be delayed (if file system is involved)
  - Cleanup needed via file system (file deletion)
- Under the hood, shared memory can be considered as an anonymous (not file-based) shared memory mapping

(File-Based) Memory Mapping

```python
import mmap
import os
from multiprocessing import Process

FILE = "shared.dat"
SIZE = 1024

def task():  1 usage
    # Child maps the file
    with open(FILE, "r+b") as f:
        mm = mmap.mmap(f.fileno(), SIZE)
        mm[0:11] = b"hello world"
        mm.flush()
        mm.close()
```

Child process creates memory mapping

flush ensures data is written to disk as well (optional here)

```python
if __name__ == "__main__":
    # Create a file of the right size
    with open(FILE, "w+b") as f:
        f.truncate(SIZE)

    p = Process(target=task)
    p.start()
    p.join()

    # Parent maps the file
    with open(FILE, "r+b") as f:
        mm = mmap.mmap(f.fileno(), SIZE)
        print(mm[0:11])
        mm.close()

    os.remove(FILE)
```

Parent process creates file for memory mapping

Parent process maps file and reads it

See code no. 4

(File-Based) Memory Mapping

```python
import mmap
import os
from multiprocessing import Process

FILE = "shared.dat"
SIZE = 1024

def task():  1 usage
    # Child maps the file
    with open(FILE, "r+b") as f:
        mm = mmap.mmap(f.fileno(), SIZE)
        mm[0:11] = b"hello world"
        mm.flush()
        mm.close()
```

Allows read and write to existing file

```python
if __name__ == "__main__":
    # Create a file of the right size
    with open(FILE, "w+b") as f:
        f.truncate(SIZE)

    p = Process(target=task)
    p.start()
    p.join()

    # Parent maps the file
    with open(FILE, "r+b") as f:
        mm = mmap.mmap(f.fileno(), SIZE)
        print(mm[0:11])
        mm.close()

    os.remove(FILE)
```

Allows read, write and create file

Parent process maps file and reads it

File needs to be removed

See code no. 4

(File-Based) Memory Mapping

- Concern: Can I not just open files “normally” without mmap in multiple processes?
- Answer: Yes, but mmap will be faster, especially if you access the file often
- Reason
  - When we open/read/write a file regularly, every operation is associated with one or more system calls, and usually copied data
  - With mmap, we treat the file as it belongs to the RAM storage
  - In the program, we interact with the file as it is an array (no kernel involvement)
  - We read/write the data directly, and changes are seen by every process directly

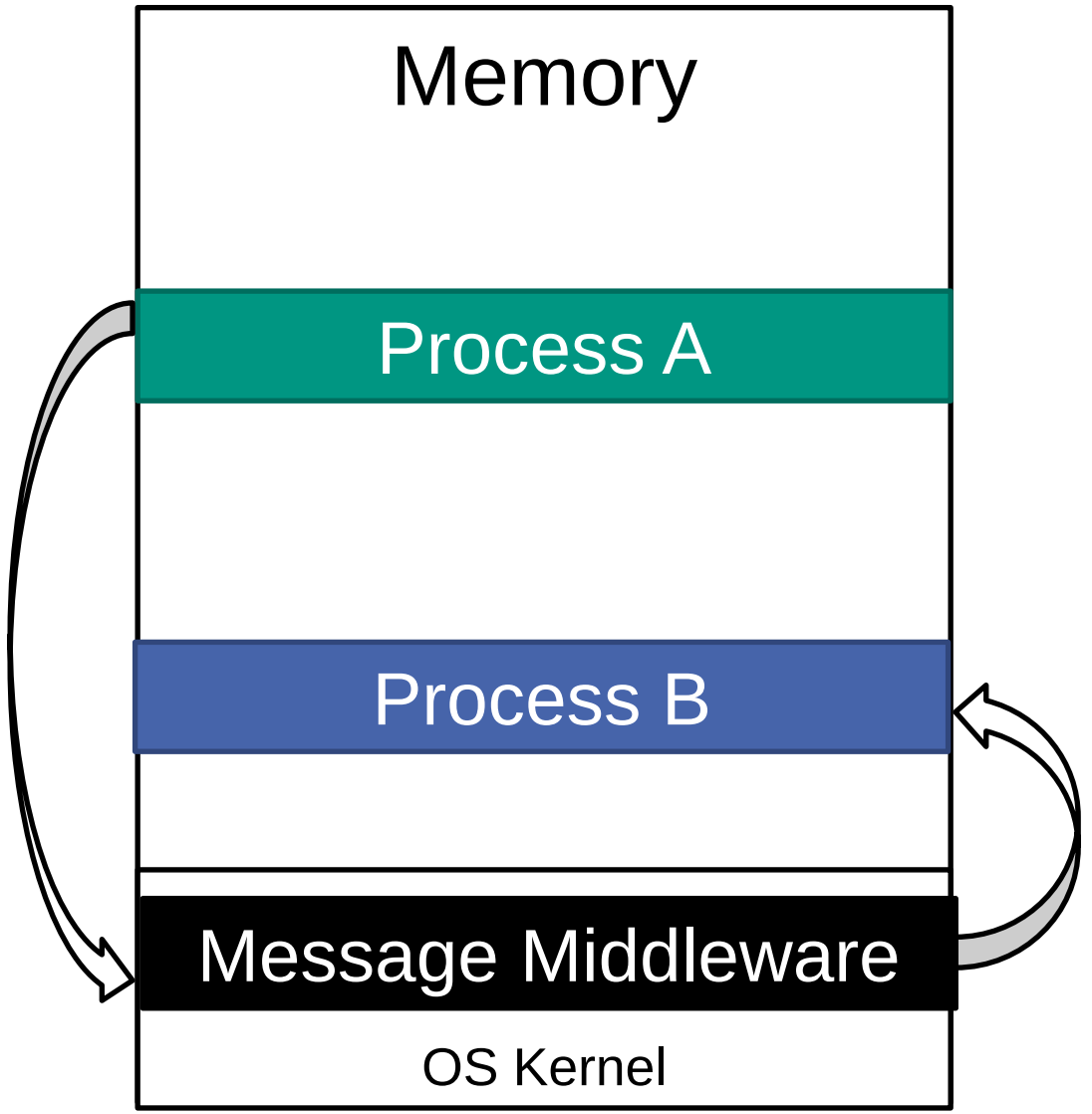
Comparison

Feature	Multiprocessing Value/Array	Multiprocessing shared_memory	mmap
Abstraction level	High	Low	Low
Data types	Can be specified	Raw bytes only	Raw bytes only
Synchronized access	Yes by default (can be deactivated)	No	No
Variable size	No	No	No, but supports resizing and large mappings (disk included)
Performance	Fast	Fastest	Fastest
Usage with other performant libraries like NumPy	Not recommended	Supported	Supported

Message-Passing IPC

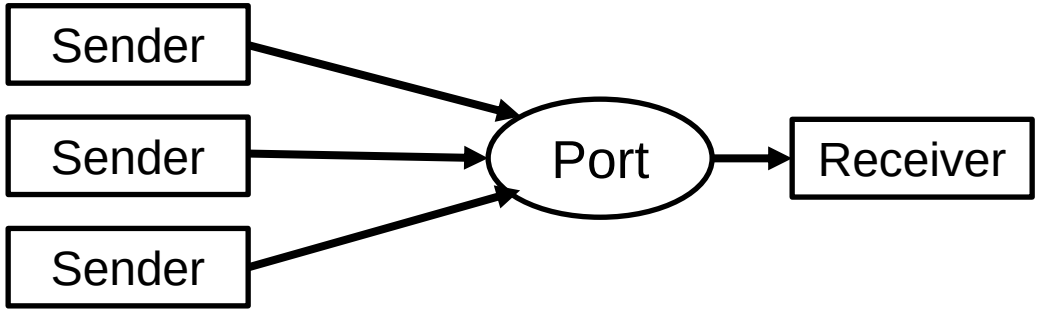
Message Passing

- Communication via sending and receiving of messages (fixed or variable size)
- Needs some „construct“ in the middle that transports messages (*middleware*)
- Message-passing IPC on one machine usually requires kernel for communication
- Implementation in high-level languages often via pipes or message queues

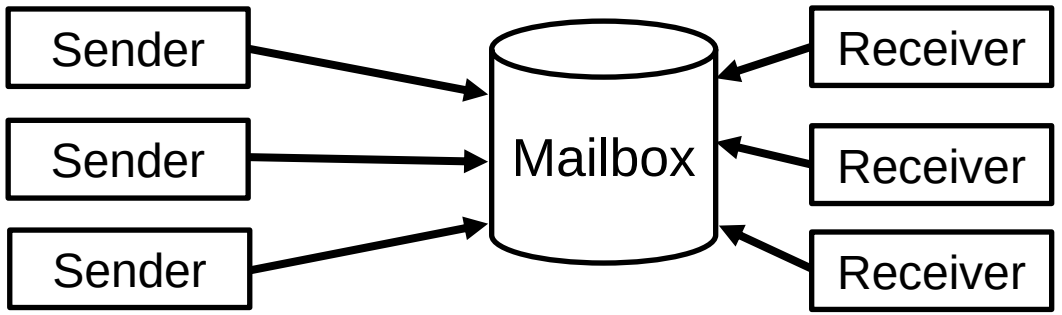


Core Concepts for Message Passing

- Port
  - Communication endpoint owned by a single receiver
  - Direct communication

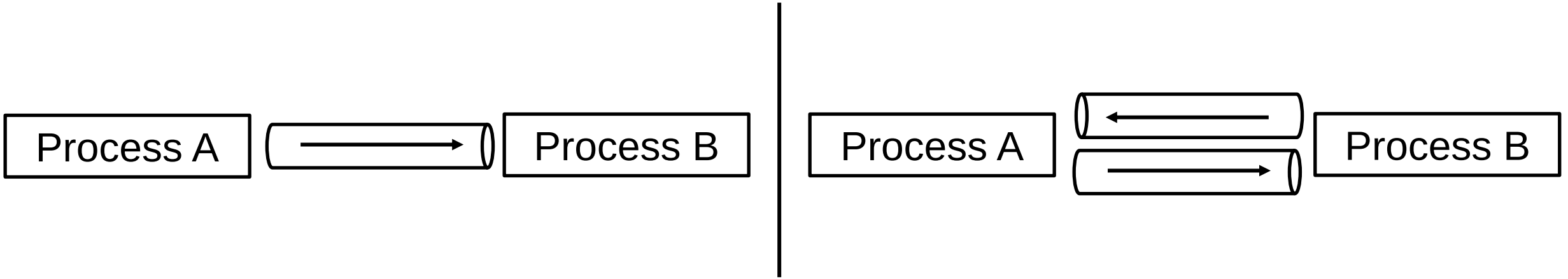


- Mailbox
  - Exists independently of senders and receivers
  - Indirect communication
  - Requires synchronization and buffering



Pipes

- Pipes are the simplest realization of message-passing IPC for two processes
- Messages of variable size can be passed
- Sent objects must be serializable
- Default in Python: bidirectional (both processes can send and receive) = two unidirectional pipes
- Can be unidirectional on request



Pipes

- Pipes use asynchronous communication
- Messages are buffered if receiver is not waiting for it already, typical buffer size is 64kB
- Buffer is managed by OS (not configurable)
- send() operation blocks if buffer is full until receiver processes messages

Unidirectional Pipe in Python

First connection is receive-only

Second connection is send-only

Creates unidirectional pipe

```
if __name__ == '__main__':
    recvConn, sendConn = Pipe(duplex=False)
    sender_process = Process(target=sender, args=(sendConn,))
    sender_process.start()
    receiver_process = Process(target=receiver, args=(recvConn,))
    receiver_process.start()
    sender_process.join()
    receiver_process.join()
```

See code no. 5

```
import time
from multiprocessing import Process, Pipe
import multiprocessing.connection as mpc


def sender(connection: mpc.Connection):  1 usage
    print('Sender: Running')
    for i in range(5):
        time.sleep(i)
        print(f'Sender sends {i}')
        connection.send(i)
    # Send stop signal
    connection.send(None)
    print('Sender: Done')


def receiver(connection: mpc.Connection):  1 usage
    print('Receiver: Running')
    while True:
        item = connection.recv()
        print(f'receiver got {item}')
        # check for stop
        if item is None:
            break
    print('Receiver: Done')
```

Bidirectional Pipe in Python

Creates
bidirectional pipe

```python
from multiprocessing import Process, Pipe
import multiprocessing.connection as mpc

def task(conn: mpc.Connection):  1 usage
    number = conn.recv()
    print(f"Child received: {number}")

    result = number ** 2

    conn.send(result)
    conn.close()

if __name__ == "__main__":
    parent_conn, child_conn = Pipe(duplex=True)

    p = Process(target=task, args=(child_conn,))
    p.start()

    parent_conn.send(5)

    result = parent_conn.recv()
    print(f"Parent received: {result}")

    p.join()
```

See code no. 6

Reading from a Pipe with Timeout

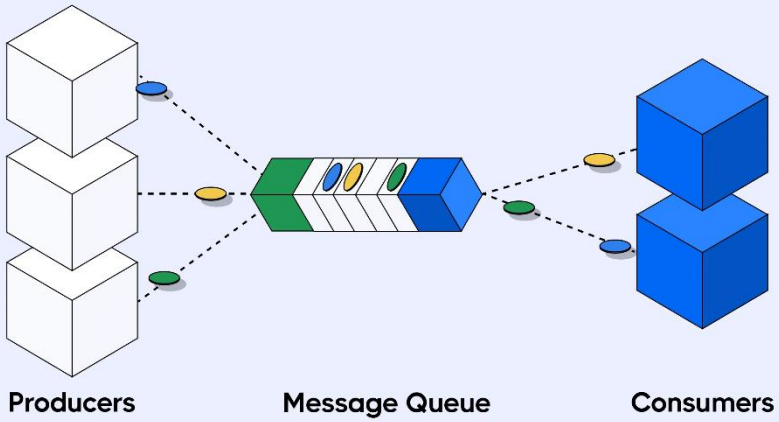
```python
import random
from multiprocessing import Process, Pipe
import time

def child_process(conn):  1 usage
    while True:
        # Wait up to 1 second for a message
        if conn.poll(1):
            msg = conn.recv()
            if msg == "DONE":
                print("Child: Exiting")
                break
            print(f"Child: Received {msg}")
        else:
            print("Child: No message, doing other work...")
            time.sleep(0.5)
```

```python
if __name__ == "__main__":
    recvConn, sendConn = Pipe(duplex=False)
    p = Process(target=child_process, args=(recvConn,))
    p.start()

    numbers = [2, 4, 6, 8]
    for num in numbers:
        print(f"Parent: Sending {num}")
        sendConn.send(num)
        sleep_time = random.randint( a: 1,  b: 5)
        print(f"Parent: Sleeping for {sleep_time} seconds...")
        time.sleep(sleep_time)

    sendConn.send("DONE")

    p.join()
```

See code no. 7+8

Message Queues

- Message queue for IPC is a linked list of messages stored in the kernel
- We send and receive messages in a first-in-first-out (FIFO) order
- Asynchronous communication
- Differences to a pipe
  - Configurable buffering, not only on OS level but also in (Python) libraries
  - Processes are fully decoupled, no connection has to be established, a process does not have to know about existence of other process
  - Some message queue implementations provide support for priority messages and enhanced error handling



Message Queues

```python
from time import sleep
import random
from multiprocessing import Process, Queue


def producer(queue: Queue):  1 usage
    print('Producer: Running')
    for i in range(5):
        value = random.randint( a: 1,  b: 5)
        sleep(value)
        queue.put(value)
    # all done
    queue.put(None)
    print('Producer: Done')
```

Put message in queue

```python
def consumer(queue: Queue):  1 usage
    print('Consumer: Running')
    while True:
        item = queue.get()
        if item is None:
            break
        print(f'Consumer got {item}')
    print('Consumer: Done')


if __name__ == '__main__':
    queue = Queue()
    consumer_process = Process(target=consumer, args=(queue,))
    consumer_process.start()
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()
    producer_process.join()
    consumer_process.join()
```

Receive message from queue

Create queue

Pass queue to process

See code no. 9

Message Queues

```python
from time import sleep
import random
from multiprocessing import Process, Queue


def producer(queue: Queue):  1 usage
    print('Producer: Running')
    for i in range(5):
        value = random.randint( a: 1,  b: 5)
        sleep(value)
        # queue.put(value)
        # queue.put(value, block=False)
        queue.put(value, timeout=3)
        print(f'Queue size: {queue.qsize()}')
        print(f'Is queue empty: {queue.empty()}')
        print(f'Is queue full: {queue.full()}')
    # all done
    queue.put(None)
    print('Producer: Done')
```

Blocks if queue is full

```python
def consumer(queue: Queue):  1 usage
    print('Consumer: Running')
    while True:
        # item = queue.get()
        # item = queue.get(block=False)
        item = queue.get(timeout=3)
        if item is None:
            break
        print(f'Consumer got {item}')
    print('Consumer: Done')


if __name__ == '__main__':
    queue = Queue(maxsize=5)
    consumer_process = Process(target=consumer, args=(queue,))
    consumer_process.start()
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()
    producer_process.join()
    consumer_process.join()
```

Blocks if queue is empty

Create queue with maximum size

See code no. 10

Message Queues

```python
from time import sleep
import random
from multiprocessing import Process, Queue


def producer(queue: Queue):  1 usage
    print('Producer: Running')
    for i in range(5):
        value = random.randint( a: 1,  b: 5)
        sleep(value)
        # queue.put(value)
        # queue.put(value, block=False)
        queue.put(value, timeout=3)
        print(f'Queue size: {queue.qsize()}')
        print(f'Is queue empty: {queue.empty()}')
        print(f'Is queue full: {queue.full()}')
    # all done
    queue.put(None)
    print('Producer: Done')
```

Exception if queue is full

Methods for queue state

```python
def consumer(queue: Queue):  1 usage
    print('Consumer: Running')
    while True:
        # item = queue.get()
        # item = queue.get(block=False)
        item = queue.get(timeout=3)
        if item is None:
            break
        print(f'Consumer got {item}')
    print('Consumer: Done')


if __name__ == '__main__':
    queue = Queue(maxsize=5)
    consumer_process = Process(target=consumer, args=(queue,))
    consumer_process.start()
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()
    producer_process.join()
    consumer_process.join()
```

Exception if queue is empty

See code no. 10

Message Queues

```python
from time import sleep
import random
from multiprocessing import Process, Queue


def producer(queue: Queue):  1 usage
    print('Producer: Running')
    for i in range(5):
        value = random.randint( a: 1,  b: 5)
        sleep(value)
        # queue.put(value)
        # queue.put(value, block=False)
        queue.put(value, timeout=3)
        print(f'Queue size: {queue.qsize()}')
        print(f'Is queue empty: {queue.empty()}')
        print(f'Is queue full: {queue.full()}')
    # all done
    queue.put(None)
    print('Producer: Done')
```

Exception if queue is full after 3 sec

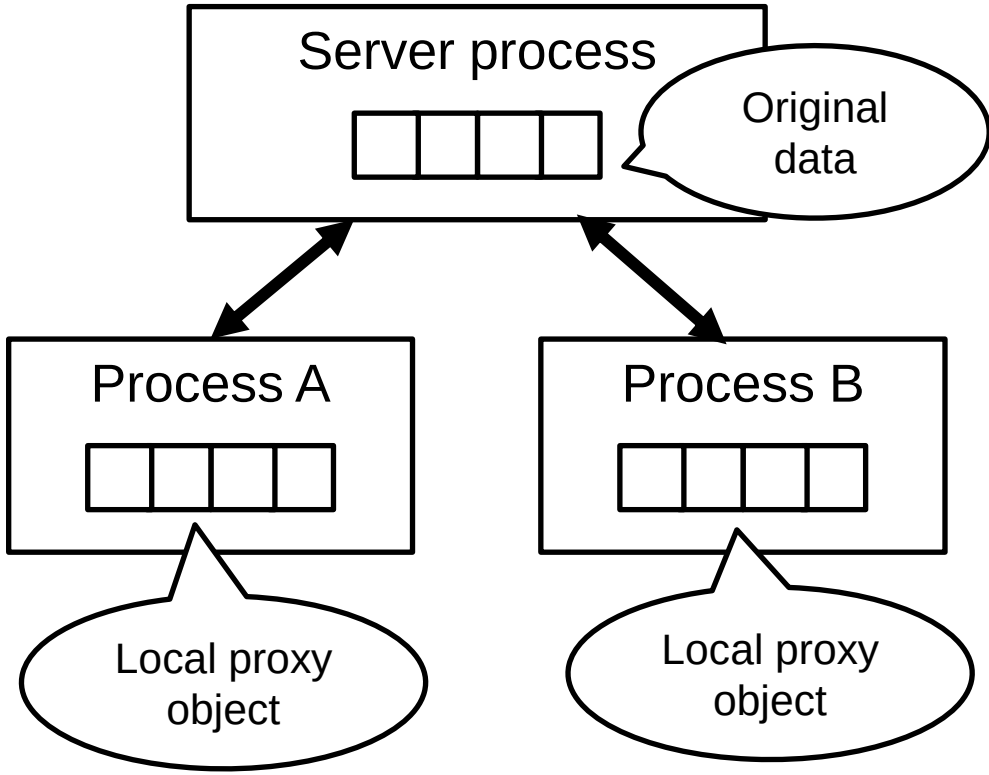
```python
def consumer(queue: Queue):  1 usage
    print('Consumer: Running')
    while True:
        # item = queue.get()
        # item = queue.get(block=False)
        item = queue.get(timeout=3)
        if item is None:
            break
        print(f'Consumer got {item}')
    print('Consumer: Done')


if __name__ == '__main__':
    queue = Queue(maxsize=5)
    consumer_process = Process(target=consumer, args=(queue,))
    consumer_process.start()
    producer_process = Process(target=producer, args=(queue,))
    producer_process.start()
    producer_process.join()
    consumer_process.join()
```

Exception if queue is empty after 3 sec

See code no. 10

Multiprocessing Manager

- The multiprocessing library provides the Manager class, a high-level abstraction for message-passing IPC
- Supports complex data structures like dictionaries and classes
- How it works
  - A server process is spawned that holds the data
  - When the data is changed, changes are submitted to server process
  - Server process interacts with other processes via pipes or sockets



Multiprocessing Manager

- Example: Process A appends new entry to a shared dict
  - Process A serializes the new entry and sends it to the server process
  - Server process appends new entry and sends updates to all processes
- Pros
  - Clear code
  - Support for complex data strcutures
  - Good for small or medium sized data
- Cons
  - Slow operations
  - Bad for large data
  - Bad for high-frequent updates
  - Manual synchronization needed for complex operations

Multiprocessing Manager

Creation of the manager

Creation of the shared dict

Passing the dict object to other process

```python
if __name__ == "__main__":
    with Manager() as manager:
        status_dict = manager.dict()
        processes = []
        for i in range(5):
            processes.append(Process(target=task, args=(i, status_dict)))

        for p in processes:
            p.start()

        for i in range(10):
            print(status_dict)
            time.sleep(0.5)

        for p in processes:
            p.join()

        print(status_dict)
```

Reading the shared dict

```python
from multiprocessing import Process, Manager
import time
from multiprocessing.managers import DictProxy
import random

def task(task_id: int, status_dict: DictProxy[int, str]):
    status_dict[task_id] = "running"
    time.sleep(random.randint( a: 1,  b: 5))
    status_dict[task_id] = "done"
```

Writes to the shared dict

See code no. 11

High-Performance Message Passing

- Previous message-passing IPC mechanisms are comparatively slow because:
  - They heavily rely on OS kernel operations
  - They serialize data for communication
  - They copy data often
- Hence, they are not good for:
  - Large datasets
  - Distributed systems (many machines connected via network)
- MPI (Message Passing Interface) is a standard aiming to provide high-performance IPC when shared memory is not possible
- Used mainly in scientific, high-performance, or cluster computing

High-Performance Message Passing

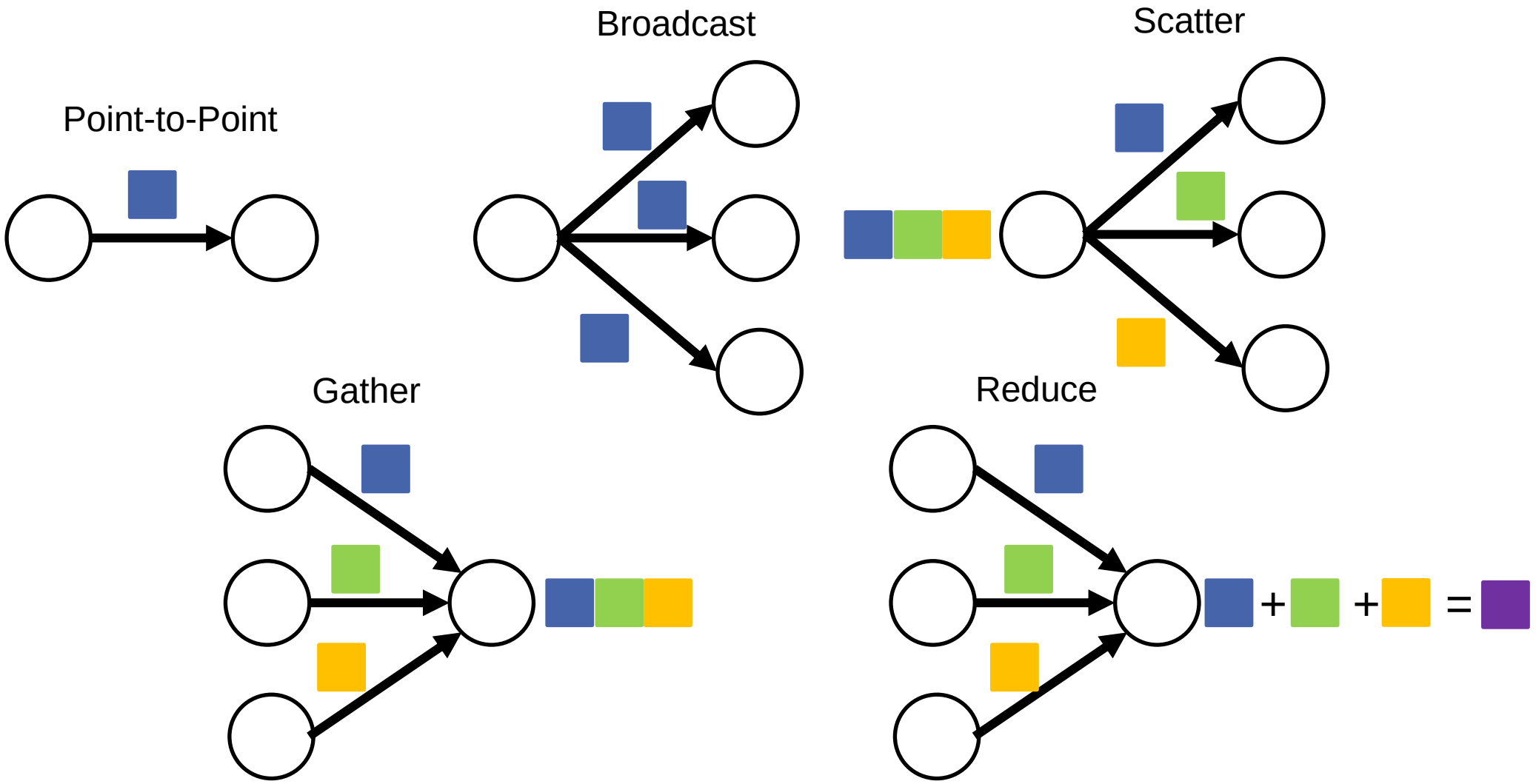
- Implementation in Python using mpi4py library
- Core concepts
  - Rank = unique ID of a process
  - Size = total number of involved processes
  - Communicator = communication context/provider
- Execution of MPI programs not directly via Python interpreter but with MPI runner
- Requires installation of MPI libraries

```python
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if __name__ == '__main__':
    print(f"Hello from rank {rank} out of {size}")
```

```
mpirun -n 4 python 12_MPIHelloWorld.py
```

See code no. 12

Supported Communication Forms in MPI

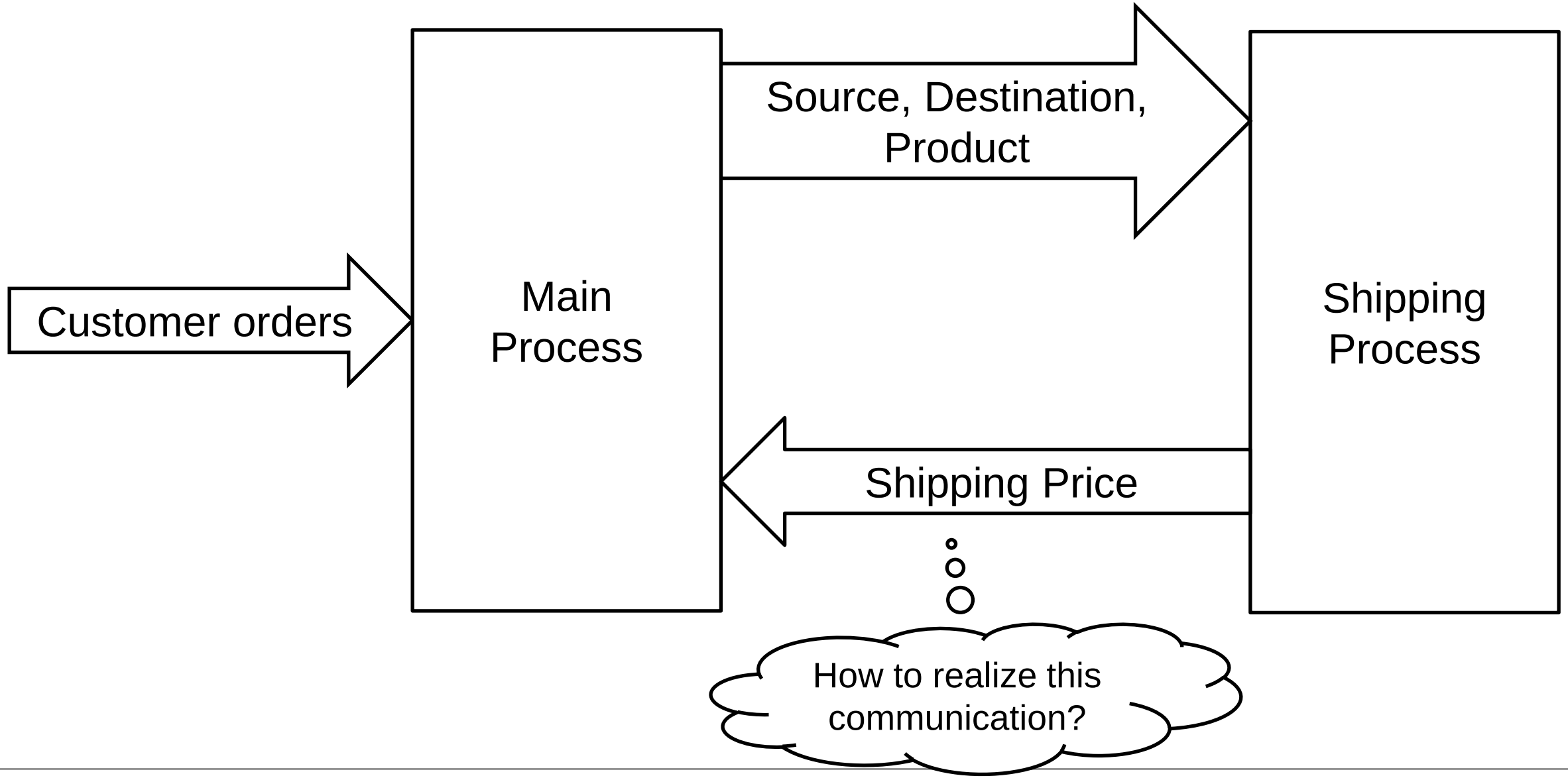


High-Performance Message Passing

- MPI is fast because
  - The interaction with the kernel is minimal
  - Data is not serialized, but usually submitted as binary data using efficient protocols
  - There is no shared state
  - It uses special libraries (like OpenMPI, Microsoft MPI) tuned for specific operating systems
- However, MPI is mostly used in specific domains, not general purpose because
  - Specific libraries need to be installed
  - Performance gains are significant only if we have complex communication patterns
  - It provides no fault tolerance (one process crashes the whole system)

IPC Design

Example Application



IPC Design Decisions: Things to Consider

- Environment: Are the processes running on the same machine or different machines?
  - Same machine: All options are on the table
  - Different machines: Limited choice, likely some message-passing IPC
- Desired features: What are features important for my system?
  - Isolation/Security: Processes should be as isolated as possible
  - Performance: Communication should be as fast as possible
  - Code simplicity: Code should be easy to understand and maintain

IPC Design Decisions: Things to Consider

- Communication: What should the communication look like?
  - Sync vs. async
  - One-way vs. two-way (request-response)
  - Point-to-point vs. 1:N vs. N:1 vs. N:N
  - Communication frequency
  - Size of transferred data

Shared Memory vs. Message Passing

- Shared memory model pros
  - Fast communication (direct memory access)
  - Efficient for large data transfers (no need to copy in every address space)
  - Kernel not involved once shared memory has been created
- Shared memory model cons
  - Complex synchronization / potential race conditions
  - Security risks
  - Manual cleanup
  - Cannot be applied to distributed systems
  - Sometimes portability issues between different operating systems

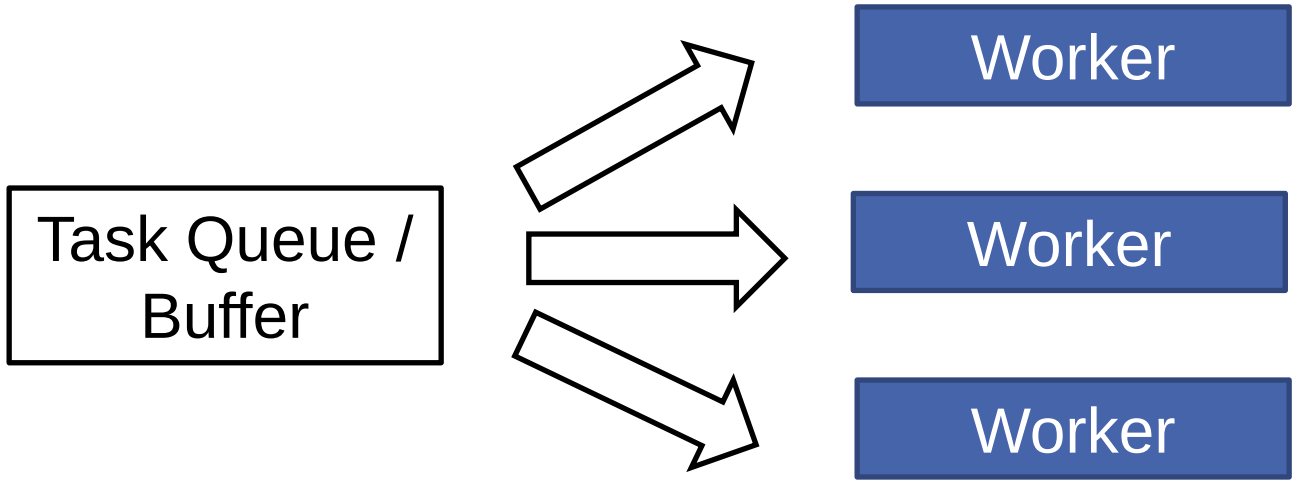
Shared Memory vs. Message Passing

- Message passing pros
  - Simple (hidden memory access)
  - No explicit synchronization needed
  - High isolation / security
  - Can be applied to distributed systems
- Message passing cons
  - Kernel involvement
  - Slow communication (data copies)
  - High resource consumption for large messages and middleware

Design Patterns: Producer - Consumer

- Producers create data/tasks
- Consumers process data/tasks
- A buffer/queue separates producers and consumers
- Producers don’t know consumers and vice versa
- Question: What if consumer is much slower than producer?
- Often realized via message queues

Design Patterns: Worker Pools



- Idea: Let a fixed number of workers process incoming tasks from a queue
- Is a worker is idle, it fetches new task from queue
- Advantage: Resources used for parallelism can be controlled
- Real-world example: Web servers

Multiprocessing Pool

```python
from multiprocessing import Pool
import os


def process_file(filename):  1 usage
    with open(filename, 'r') as f:
        return sum(1 for line in f)


if __name__ == "__main__":
    files = [f for f in os.listdir('.') if os.path.isfile(f)]

    with Pool(os.cpu_count()) as p:
        line_counts = p.map(process_file, files)

    print(line_counts)
```

Count lines in file

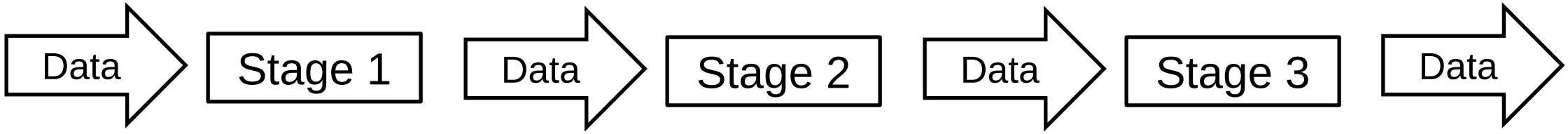
Get list of files

Create process pool

Pool should execute this function on the file list

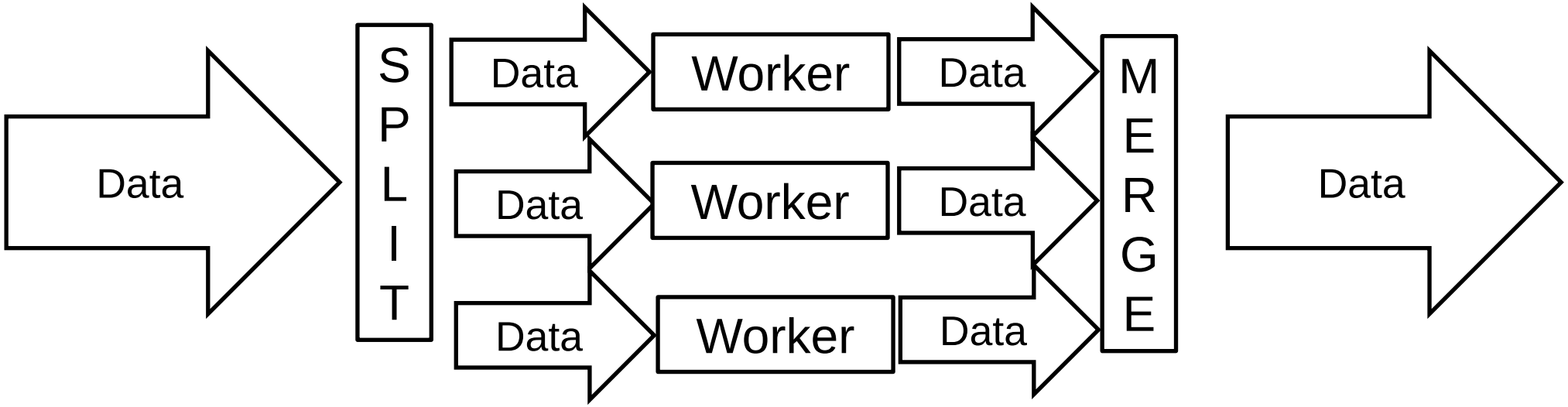
See code no. 13

Design Patterns: Pipeline



- Often used for processing data streams
- Process divided into stages that can be executed concurrently
- Communication realized with buffers/queues
- Example: Image processing
  - Stage 1: Adjust size
  - Stage 2: Optimize colors
  - Stage 3: Apply filter

Design Patterns: Fan-Out, Fan-In



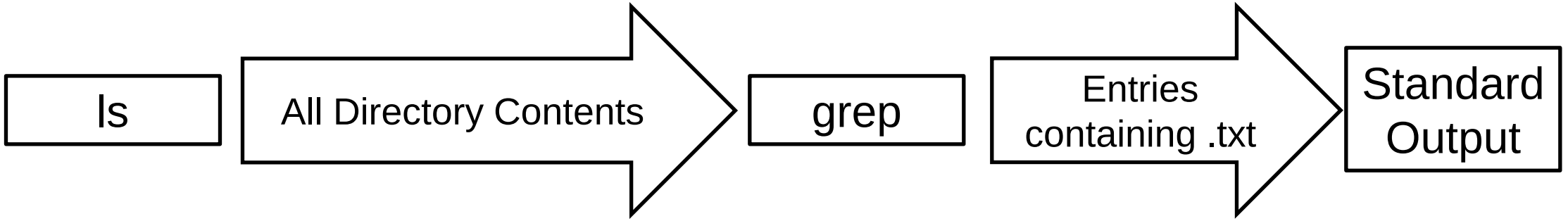
- Idea: Split huge amount of data into chunks that be processed in parallel (Fan-Out)
- After processing, merge data again (Fan-In)
- Often combined with pipeline pattern
- Real-world examples:
  - Sorting large arrays
  - MapReduce

IPC Use Cases

Use Case: Video Streaming

- Video is stored in compressed and decoded mp4 format
- One process (decoder) processes file and writes decoded data to shared memory, second process (renderer) shows it on display
- Shared memory used for high-frequency, low-latency, large data transmission

Use Case: Linux Pipe




```
ls | grep ".txt"
```

- Output of one command (process) is streamed via a pipe to another command (process)
- Here: ls lists the contents of a directory, grep filters which entry is containing .txt

Use Case: Airport

- Real-time flight information visualized on screens
- Many producers (e.g., different airlines) update their flight status
- “Fire-and-forget” communication
- Realized via message queues